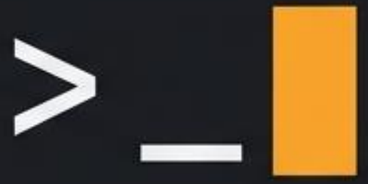
Engineering the AI Terminal



A Blueprint for Claude Code Customization

System design, token economics, and isolation boundaries for file-native AI assistants.

Moving from chat boxes to version-controlled infrastructure

The IDE Chat Paradigm (Legacy)

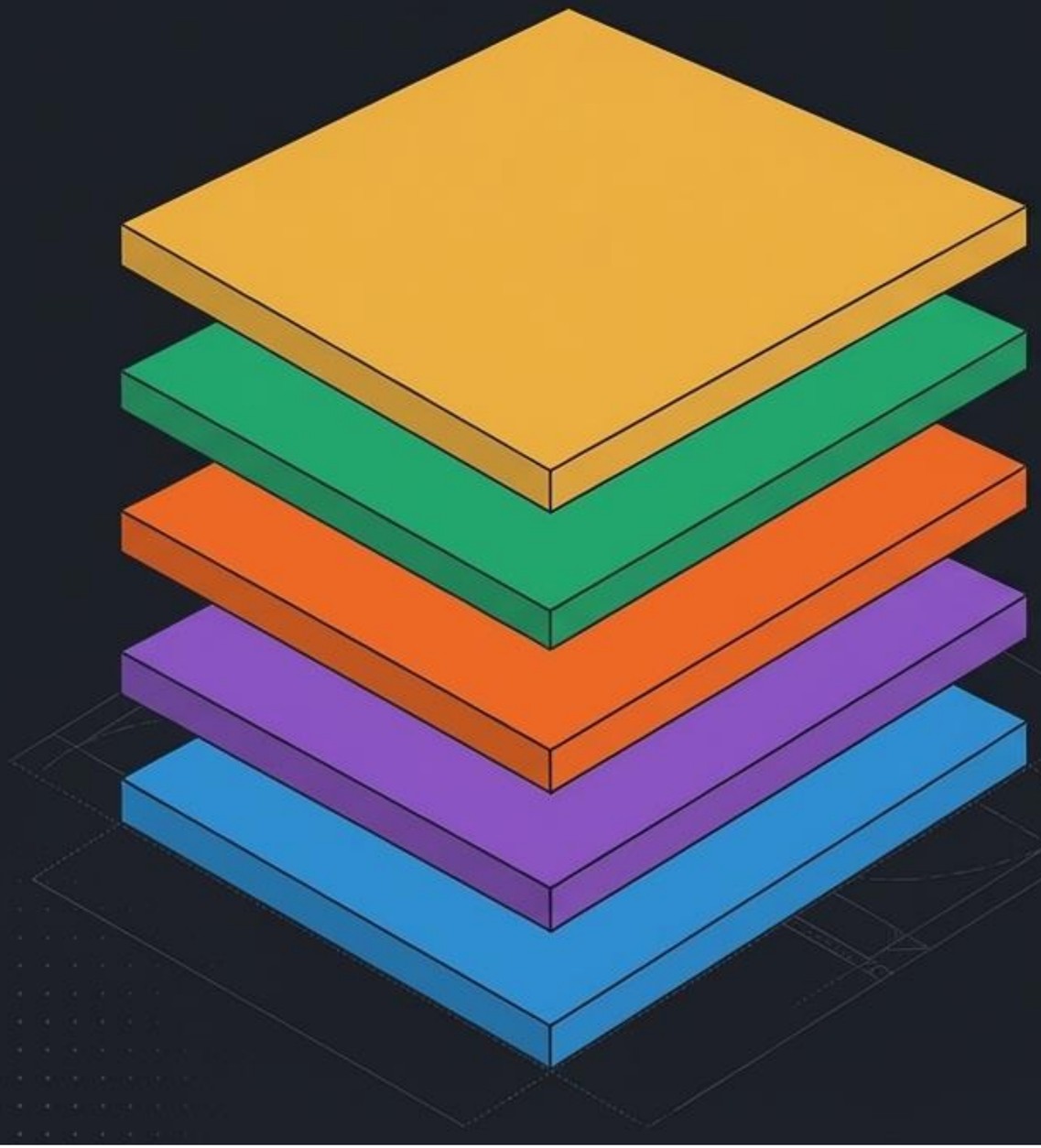
- 🔍 - Unstructured, infinite context accumulation
- 🧠 - UI-bound configurations
- 🔌 - Black-box behavior and non-deterministic guardrails
- 🛡️ - Prone to context bloat and instruction drift

The File-Native Terminal (Claude Code)

- 📄 - All artifacts are plain text files (.md, .json)
- 🔀 - Version-controlled and team-shared by default
- >_ - Observable stdin/stdout boundaries
- 📚 - Strict, layered execution environments

Philosophy: Every context decision must be observable, auditable, and engineered.

The Five-Layer Cooperative Architecture



Commands: The Workflow Layer.
User-invoked prompt macros for repeatable tasks.
JetBrains Mono: `.claude/commands/*.md`

MCP Servers: The Tool Layer.
External capabilities, APIs, and file systems.
JetBrains Mono: `settings.json -> mcpServers`

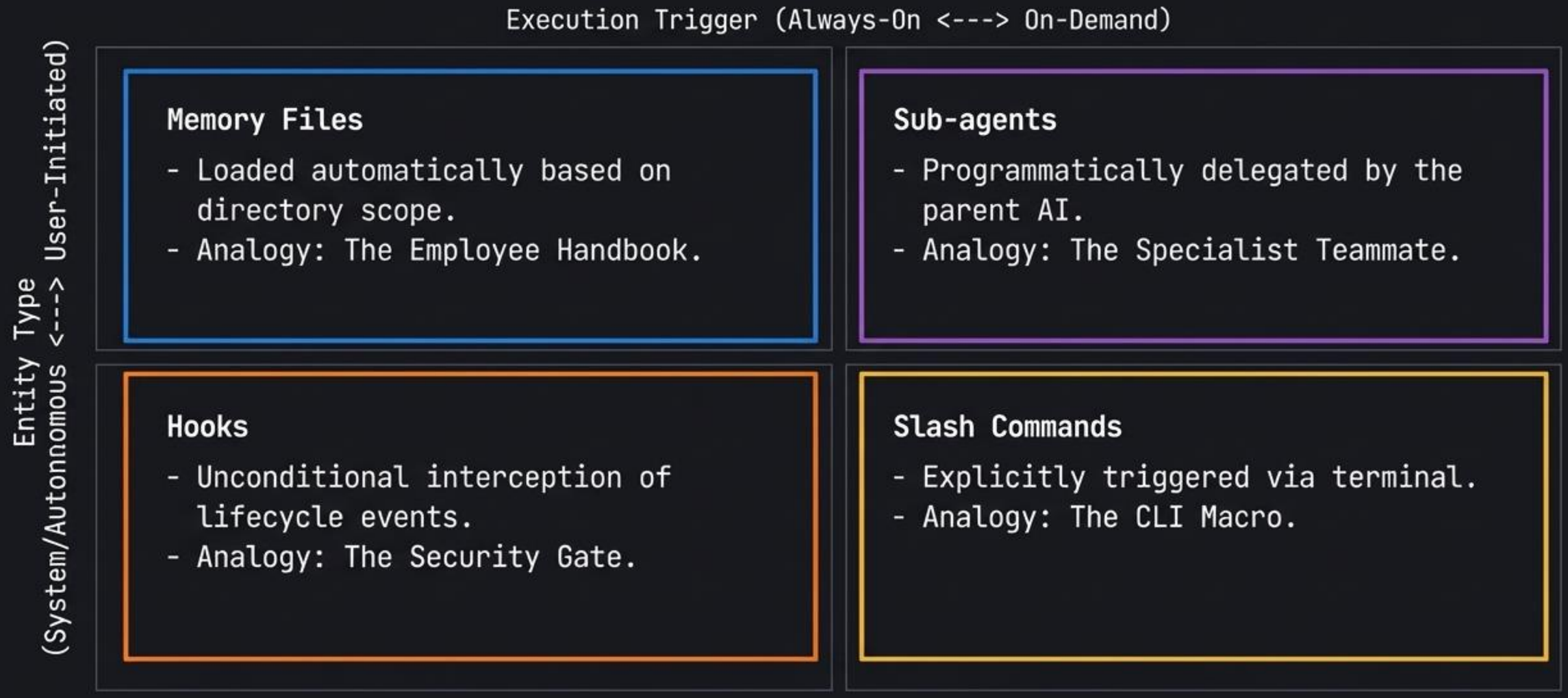
Hooks: The Safety Layer.
Deterministic security checkpoints and automation triggers.
JetBrains Mono: `settings.json -> hooks`

Sub-agents: The Role Layer.
Specialized, isolated workers invoked on-demand.
JetBrains Mono: `.claude/agents/*.md`

Memory: The Context Layer.
Always-on project conventions and stack preferences.
JetBrains Mono: `CLAUDE.md`

Claude Code behaves like a disciplined engineering team: Memory establishes what we know, Agents define who does the work, Hooks ensure it's done safely, and MCP decides what tools we have.

Mapping the execution triggers and autonomy boundaries



(Note: MCP Servers span the X-axis as they load at session start but are invoked on-demand).

The Memory Hierarchy: Merging context without contradictions

1. Sub-directory Memory
`(src/CLAUDE.md)`
Module-specific rules.
Loaded only in subtree.

2. Local Project Memory
`(.claude/CLAUDE.md)`
Machine-specific local paths.
Not committed.

3. Project Memory
`(CLAUDE.md)`
The core team conventions.
Always-on within repo.

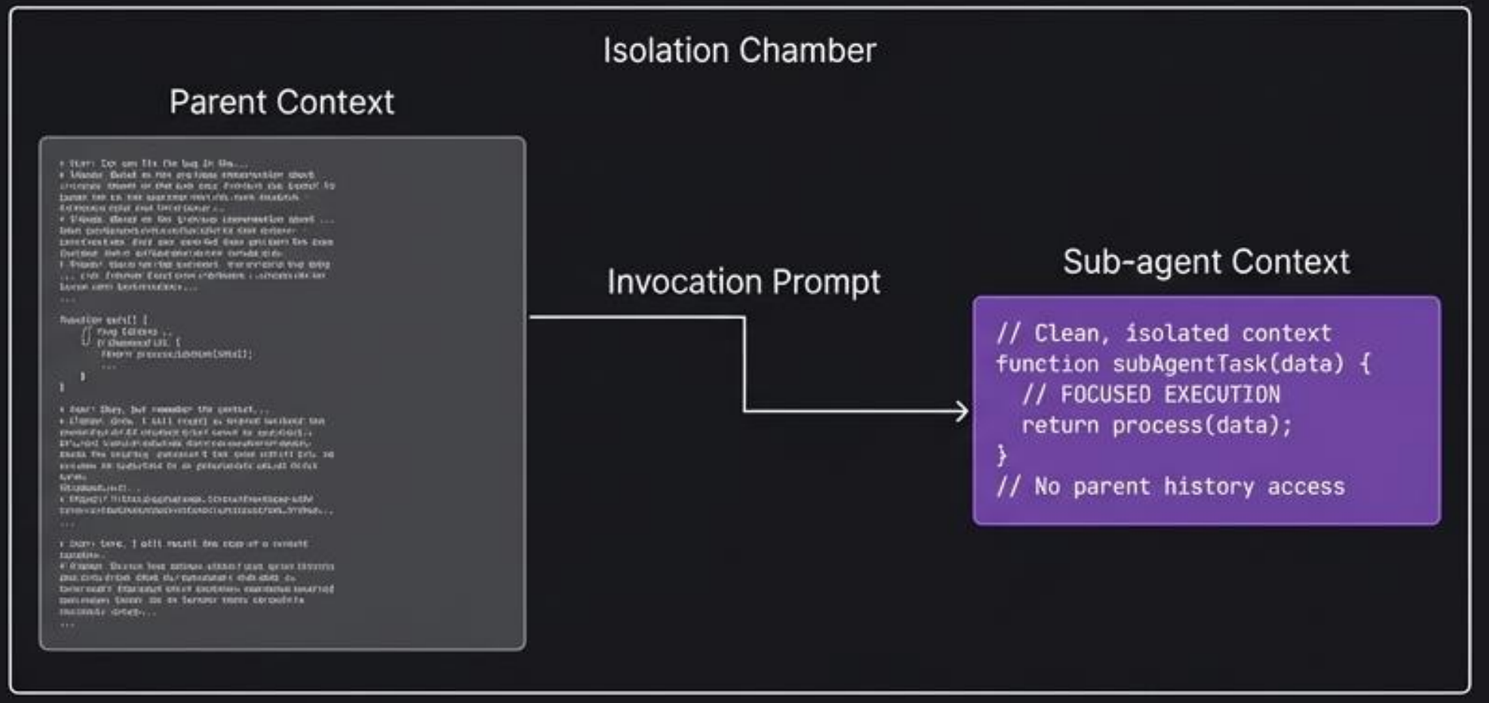
4. User Memory
`(~/.claude/CLAUDE.md)`
Global editor habits.
Always-on across projects.

Merged Context
Deterministic combination of all applicable rules.

The Engineer's Guideline

- → **Target Size:** 5-20 rules per file (200-800 tokens max).
- → **Inclusion:** Tech stack, architectural decisions, and rejected anti-patterns.
- → **Exclusion:** Do NOT embed complex workflow orchestration. Lengthy runbooks dilute the baseline signal.

Sub-agents run in strict isolation to prevent context bleed



The Architectural Constraint: Single-Level Delegation

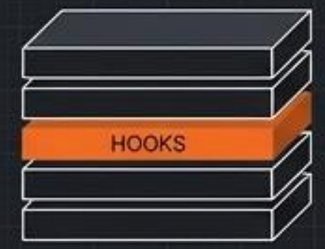
Sub-agents cannot invoke other sub-agents. The hierarchy is strictly one level deep. This prevents infinite recursion loops and unbounded token consumption.

Why Context Isolation Matters:

1. Zero History Bloat: No inheritance of parent conversation history.
2. Security by Default: Cannot exfiltrate unprovided context.
3. Hyper-Focus: Context window entirely dedicated to the delegated task.

`Crucial Configuration: description: The most critical frontmatter field. Claude uses this exact text to autonomously decide when to invoke the sub-agent.`

Hooks: Deterministic interception at zero token cost



Security Checkpoint

AI Process
(Tool Call)

Outputs Exit Code 0
Gate opens. Operation proceeds.

File System / OS

Hook Checkpoint
(stdin/stdout)

Outputs Exit Code 2
Gate slams shut. Immediate bounce-back to Claude as "Permission Denied". LLM token engine bypassed.

The Interception Protocol:

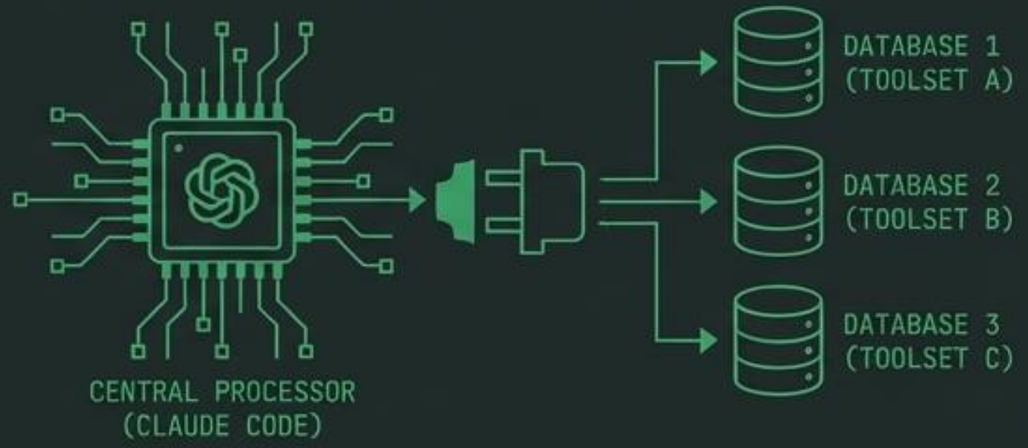
Hooks are non-LLM, deterministic shell scripts executing at lifecycle points (`PreToolUse`, `PostToolUse`, `Stop`). They receive a JSON payload via `stdin` and write to `stdout`.

Why Principal Engineers Love Hooks:

Because they run at the **OS process level**, hooks consume **0 tokens**. They are the only architecturally correct location for unconditional enforcement (e.g., blocking `rm -rf`).

Extending capabilities and formalizing explicit workflows

MCP Servers (The Interface)

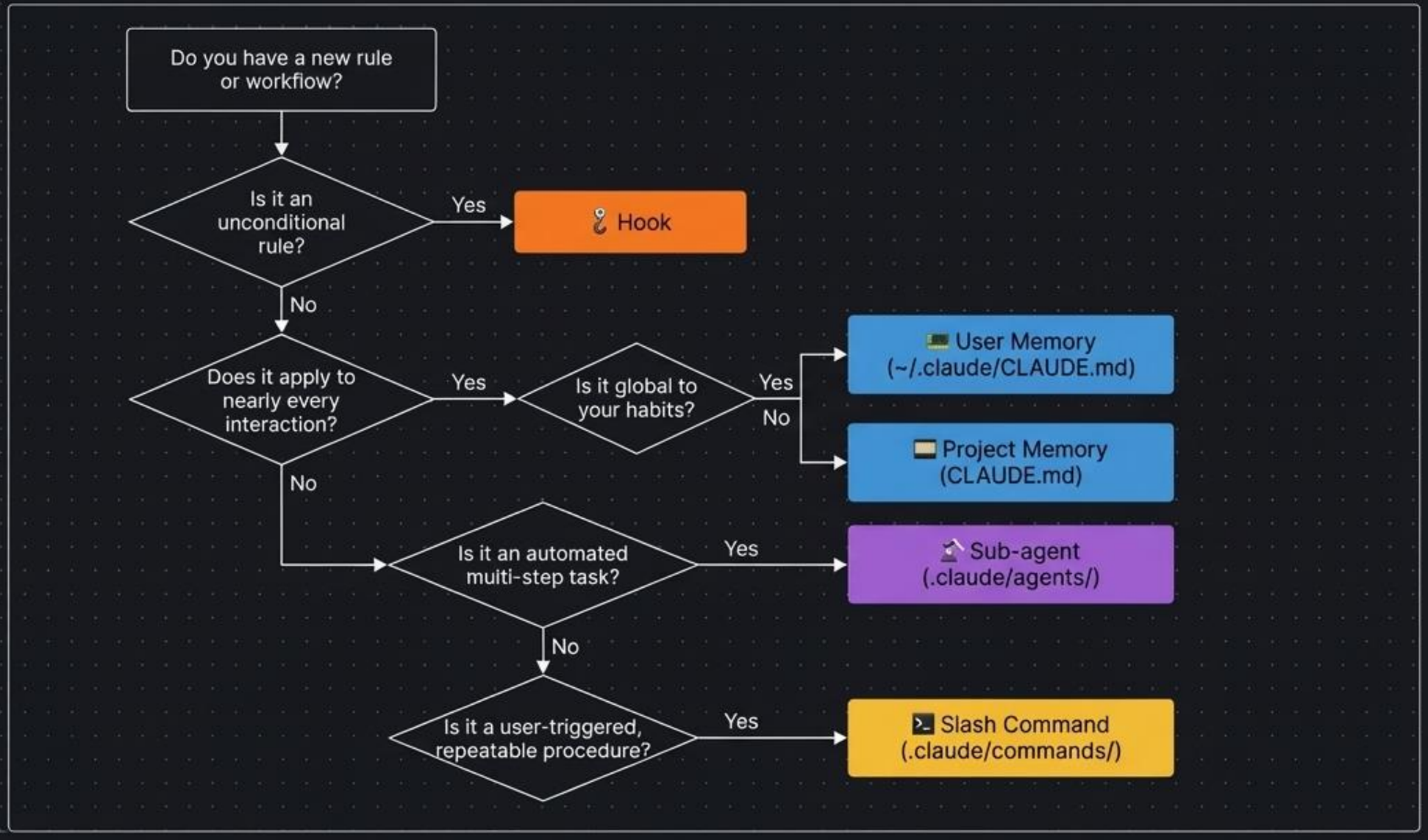


- **Role:** Claude Code acts as an MCP client; servers provide external toolsets.
- **Transports:** `stdio` (local binaries), `http` (cloud APIs), `sse` (real-time data).
- **Security Rule:** Never hard-code credentials. Scope `mcpServers` in `settings.json` to the minimum required database or path.

Slash Commands (The Trigger)

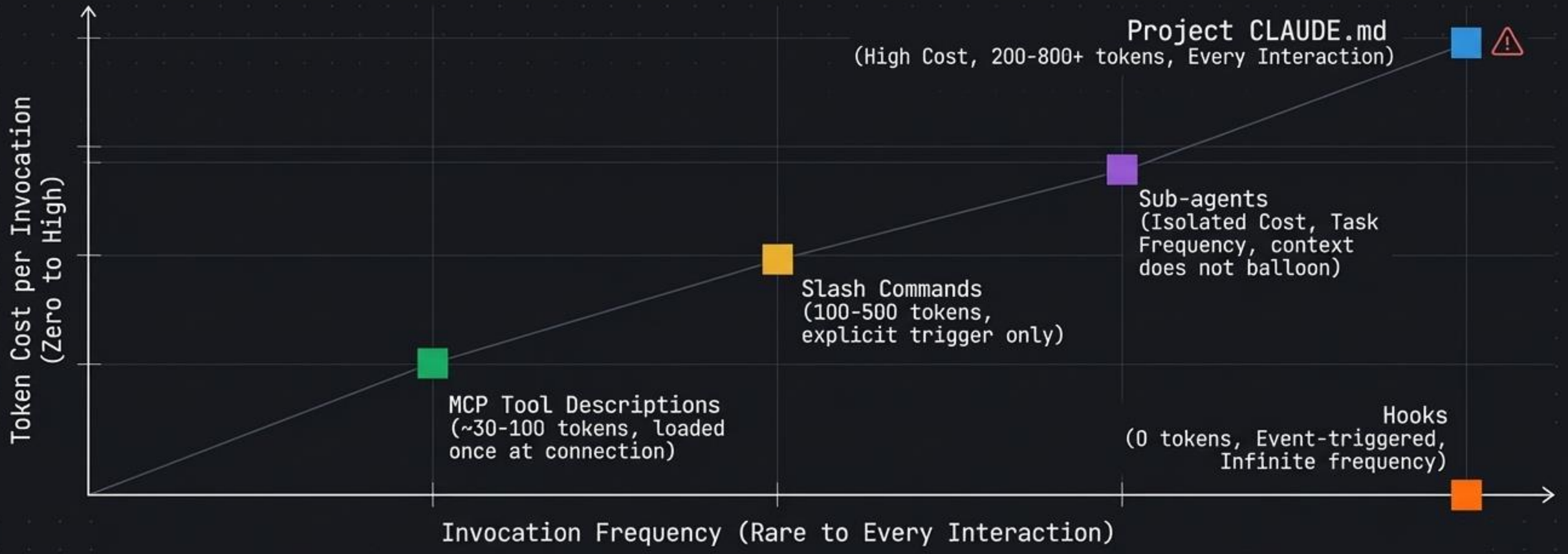
- **Role:** Reusable, parameterized prompt macros (`.claude/commands/*.md`).
- **Trigger:** User-invoked (`/project:<name>`).
- **The Power Feature:** Commands support `$ARGUMENTS` for dynamic input and can explicitly instruct the parent to orchestrate multiple sub-agents.

Where does the workflow belong? A Placement Framework



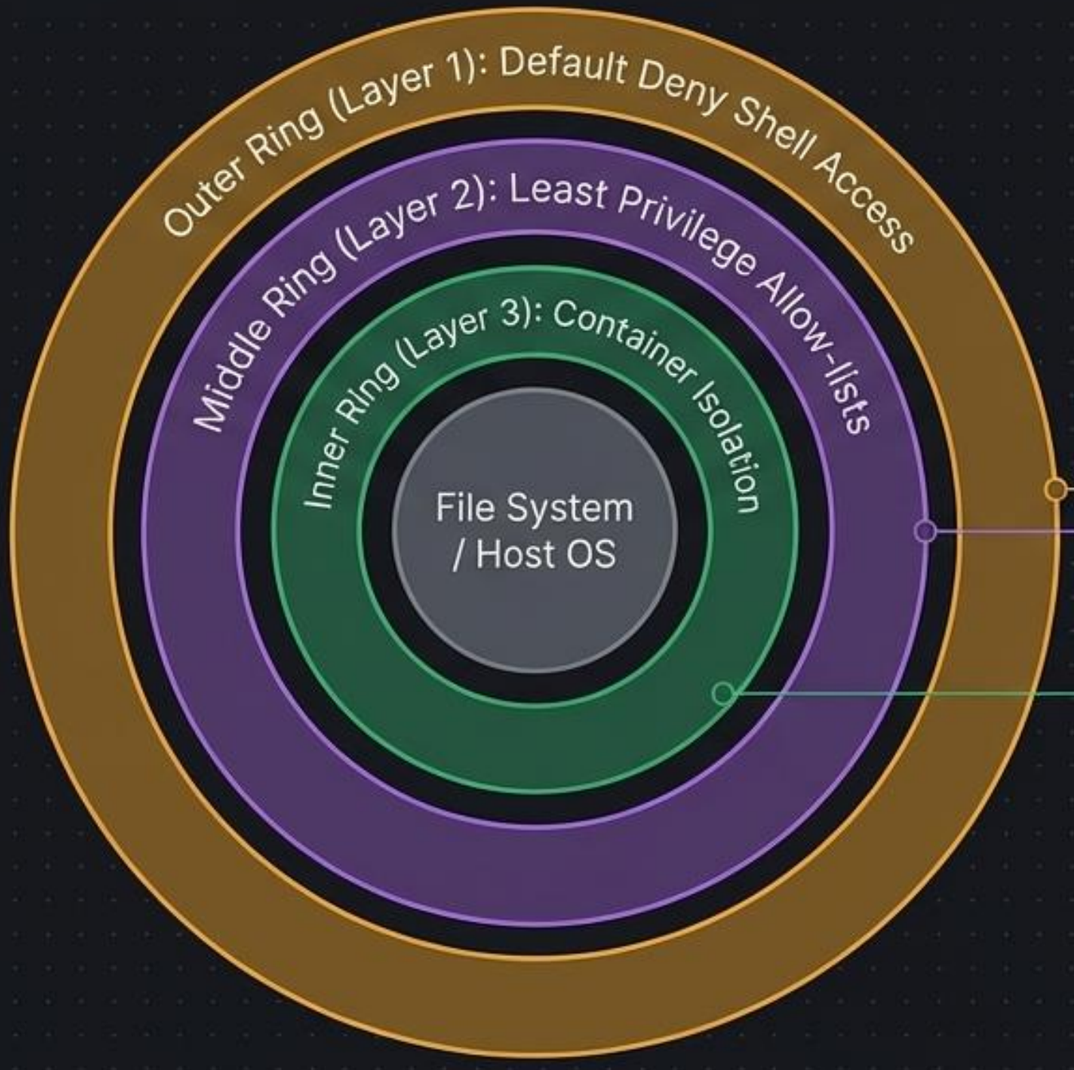
The Anti-Pattern Warning: Writing an entire CI debugging runbook in CLAUDE.md wastes tokens on every simple question and dilutes context.

Token Economics: Architecting for cost and scale



Strategic Principle: Never always-on what can be on-demand. Sub-agents are cheaper than context accumulation.

The Defense Model: Zero-trust execution boundaries



Layer 1: Default Deny Shell Access

Explicitly enumerate allowed commands in .claude/settings.json. The final rule must be a catch-all block:

```
Bash(pnpm test -- *) -> Allowed
Bash(*)              -> Blocked (Deny rule)
```

Layer 2: Least Privilege Sub-agents

Use 'tools' (whitelist) and 'disallowedTools' (blocklist) frontmatter to restrict access. Example: A Reviewer agent has no write access.

Layer 3: Container Isolation

Route agents executing builds, tests, or DB migrations through a containerized MCP server, isolating the host OS.

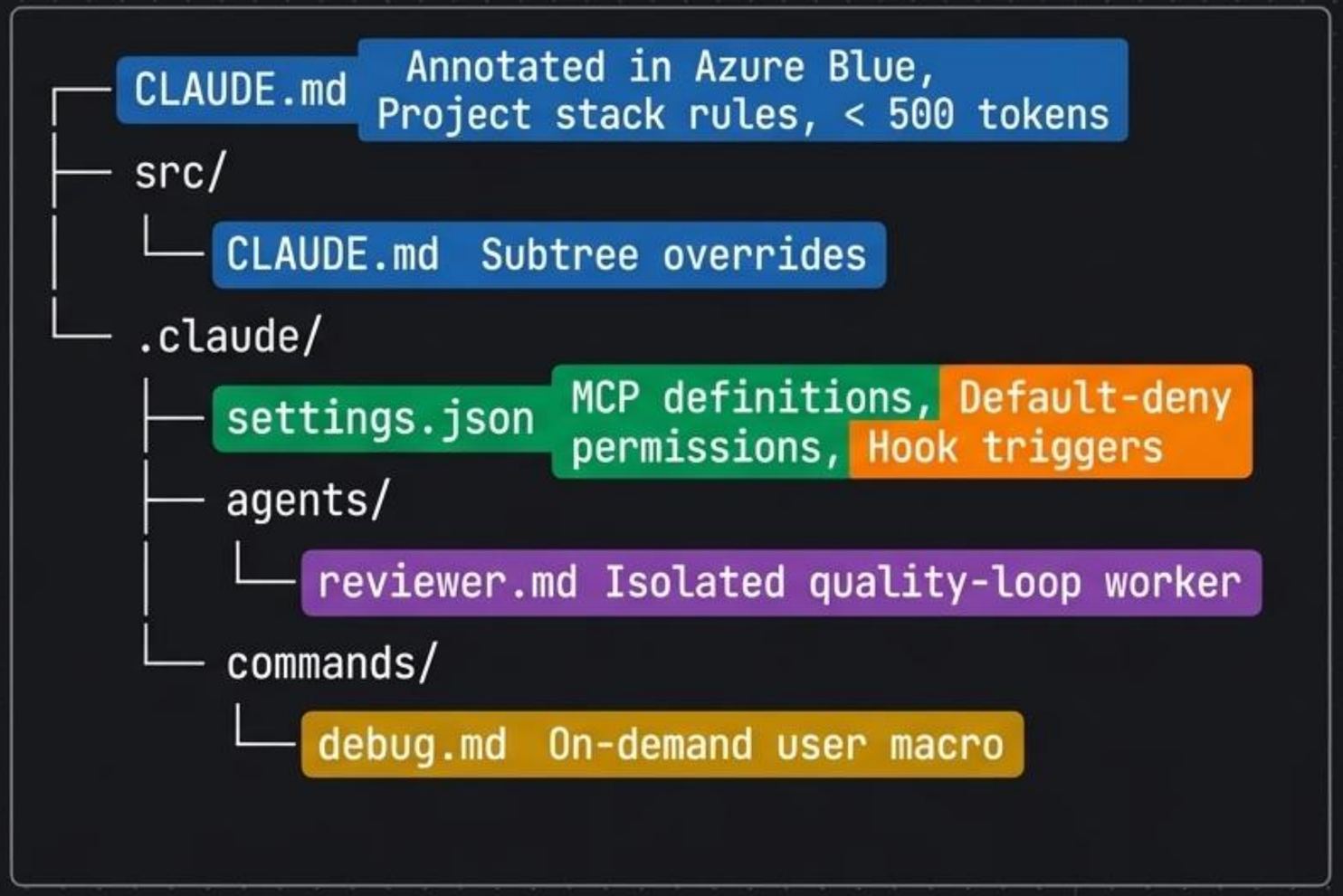
⚠ Audit Rule: Any sub-agent using permissionMode: bypassPermissions requires documented justification and is prohibited in production pipelines.

Architectural Benchmark: Claude Code vs. GitHub Copilot

Dimension	Claude Code (Terminal-First)	GitHub Copilot (IDE-First)
Primary Configuration	`CLAUDE.md` (Version-controlled)	`.github/copilot-instructions.md`
Sub-agent Location	**Plain text** `.md` in `.claude/agents/`	`.agent.md` in `.github/agents/`
Hook Granularity	**Tool-level** (`PreToolUse` per call)	**Event-level** (git, session)
Hook Interception	**Exit code** 2 blocks execution via stdout	`permissions.allowedCommands`
Token Optimization	Sub-agents run in isolated contexts	Sub-agents run in isolated contexts
Tool Integration	**Native MCP client** via `settings.json`	VS Code Settings dependent

Core Difference: Claude Code emphasizes deterministic script enforcement (Hooks) and plain-text **terminal parity across all platforms**, rather than relying on IDE-specific guardrails.

The Blueprint Complete: A disciplined, file-native assistant



Final Takeaway:

> Claude Code is not a magic chat box.

> It is a version-controlled, auditable component of your team.

> Constrain its context.

> Enforce rules with zero-token hooks.

> Delegate complex tasks to isolated agents.